ASSIGNMENT 1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Name: A. Midhuna

Register no: 192521302

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)

Nature of Assignment: Individual

Duration: 1 Week

Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5

level) Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link:

_____ How to Answer the

Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Namma Metro Smart-Card Fare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least
Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version of the gate software was written in a hurry. Some fares are off by a paisa, the peak check accepts 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance once appeared on a card, and the fare values are typed in several places.

The number of journeys processed may grow:

1 station (20,000 journeys a day) → 1 line (500,000 a day) → whole network (3,000,000 a day)

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants, operators and control structures, implement the fare logic and validate it, so that every fare and balance is exact.

Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects 4 Marks

Bloom's Level: K5 – Evaluate

Analyse the fare problem from the object-oriented point of view.

Discuss:

• The objects in the problem (card, passenger, journey, zone, fare table) and the data and behaviour of each

• How the same problem would be written in a procedural style, and what changes in an object-oriented style

• The basic principles of OOP – encapsulation, abstraction, inheritance and polymorphism – and where each could appear in this system

• The rules every correct journey must satisfy (for example: balance never below Rs 50 after a journey)
You are expected to justify each object and rule, not merely list names.

Alternative Concept and Complexity Analysis

If the fare system were written in a purely procedural style (global variables and free functions) instead of SmartCard, Journey and FareTable classes, what would the solution be? Analyse whether the time per journey or the memory per card changes (both are O(1)), and compare how much code must be changed when one fare rule changes.

Evidence Expected

A clear object-oriented analysis with objects, their data and behaviour, rules and stated assumptions.

ANSWER:

Understand the Problem and Identify the Objects

Object-Oriented Analysis of Fare System

An automatic fare system can be designed using Object-Oriented Programming (OOP). It manages passenger journeys, smart card balances and fare calculations.

Objects, Data and Behaviour

SmartCard

- Data: Card ID, balance and card status.
- Behaviour: Check balance, deduct fare and update balance.
- Justification: It stores passenger money and ensures the balance is maintained correctly.

Passenger

- Data: Passenger ID, name and card details.
- Behaviour: Tap the card, start a journey and complete a journey.
- Justification: It represents the person who uses the transport system.

Journey

- Data: Source zone, destination zone, journey time and fare.
- Behaviour: Calculate journey fare and validate the journey.
- Justification: It records the travel details needed to calculate the correct fare.

Zone

- Data: Zone ID and zone name.
- Behaviour: Identify the travel zone.
- Justification: It helps determine the fare based on the passenger's travel location.

FareTable

- Data: Fare values for different zones and journeys.
- Behaviour: Calculate and return the correct fare.
- Justification: It stores fare rules in one place, making fare changes easier.

Procedural Style vs Object-Oriented Style

Procedural Style:

- Uses global variables and separate functions.
- Data and functions are maintained separately.
- Example: checkBalance(), calculateFare() and deductFare().

Object-Oriented Style:

- Uses classes and objects such as SmartCard, Journey and FareTable.
- Data and related functions are combined inside classes.
- It improves code reusability, security and maintenance.

Basic Principles of OOP

- Encapsulation: Combines data and methods inside a class. For example, SmartCard keeps its balance private and allows changes through methods.
- Abstraction: Hides internal implementation details. For example, passengers only need to tap their card without knowing how the fare is calculated.
- Inheritance: Allows one class to acquire properties from another class. For example, StudentCard and SeniorCitizenCard can inherit from SmartCard.
- Polymorphism: Allows the same method to perform different operations. For example, calculateDiscount() can provide different discounts for student and senior citizen cards.

Rules for a Correct Journey

- A smart card must be valid and active.
- The passenger must have a minimum balance of Rs. 50 before starting a journey.
- The correct fare must be calculated based on the source and destination zones.
- The fare must be deducted only once for a completed journey.
- The card balance must never fall below Rs. 50 after a journey.
- Invalid journeys must not deduct any fare.

Assumption: Rs. 50 is the minimum balance that must remain after fare deduction.

Alternative Concept and Complexity Analysis

If the system is developed using a purely procedural approach, global variables can store card balance, passenger details and fare values. Separate functions can perform fare calculation, balance checking and fare deduction.

Complexity Analysis

Operation	Procedural	OOP
Time per journey	O(1)	O(1)
Memory per card	O(1)	O(1)

Both approaches have the same time and memory complexity because each journey requires a fixed number of operations.

Changes When Fare Rules Change

- Procedural approach: Changes may be required in multiple functions because fare rules and data are maintained separately.
- OOP approach: Changes can mainly be made inside the FareTable class without affecting other classes.

Conclusion:

The object-oriented approach is more suitable for the fare system because it provides encapsulation, reusability, security and easy maintenance. Both approaches have O(1) time and memory complexity, but OOP makes future modifications easier.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly 3 Marks Bloom's Level: K5 – Evaluate

Define the fares, surcharge and limits so that a revision is made in one

place only. Discuss:

• The five kinds of tokens in your program (keywords, identifiers, constants,

operators, punctuators) with examples

• #define, const and constexpr constants, and why constexpr is preferred for the fare

table • enum and enum class for zones, and why enum class is safer

• Naming rules and conventions for identifiers

Alternative Concept and Complexity Analysis

If the fares were written with #define macros instead of constexpr constants and an enum class, what would change? Compare type checking and scope, and analyse the memory used (a macro occupies no storage; a constexpr value usually does not either) and whether the running time is affected.

Evidence Expected

A single, well-named set of constants with a justified choice between #define, const, constexpr and enum class.

ANSWER:

Five Types of Tokens in C++

Tokens are the smallest individual elements of a C++ program.

- Keywords: Reserved words with special meanings. Examples: int, class, if, return, public.
- Identifiers: Names given to variables, classes and functions. Examples: SmartCard, baseFare, calculateFare().
- Constants: Fixed values that cannot be changed during program execution. Examples: 50, 100, 5.0.
- Operators: Symbols used to perform operations. Examples: +, -, *, ==, >=.
- Punctuators: Symbols used to structure the program. Examples: { }, ( ), ;, ,.

Defining Fares and Limits Using Constants

The following code defines all fare values and limits in one place.

```
#include <iostream>

using namespace std;

constexpr double BASE_FARE = 10.0;
```

```
constexpr double ZONE_SURCHARGE = 5.0;

constexpr double MIN_BALANCE = 50.0;

constexpr double MAX_FARE = 100.0;

 e num class Zone {

   Zone1,

   Zone2,

   Zone3

};
```

- #define: Creates symbolic macros but does not provide proper type checking.
- const: Declares a value that cannot be modified after initialization.
- constexpr: Defines a compile-time constant when its initializer permits it.

Why constexpr is preferred:

constexpr provides type safety, compile-time evaluation and easier maintenance. If the fare changes, we only need to update the constant in one place.

enum and enum class for Zones

- enum: Defines named integer constants, but its values can automatically convert to integers.
- enum class: Defines strongly typed constants and prevents unwanted implicit conversions.

Example:

```
enum class Zone {

   Zone1,

   Zone2,

   Zone3

};

Zone passengerZone = Zone::Zone1;
```

enum class is safer because it avoids naming conflicts and provides better type checking.

Naming Rules and Conventions

- Identifiers must begin with a letter or underscore.

- They cannot contain spaces.
- Keywords cannot be used as identifiers.
- Variable names should be meaningful.
- Constants are commonly written in uppercase.
- Class names use PascalCase.
- Function and variable names use camelCase.

Examples: SmartCard, calculateFare, cardBalance, MIN_BALANCE.

Alternative Concept and Complexity Analysis

If #define macros are used instead of constexpr and enum class, the program may look like this:

#define BASE_FARE 10.0

#define ZONE_SURCHARGE 5.0

#define MIN_BALANCE 50.0

#define MAX_FARE 100.0

#define ZONE1 1

#define ZONE2 2

#define ZONE3 3

Comparison:

Feature	#define	constexpr and enum class
Type checking	No proper type checking	Strong type checking
Scope	Preprocessor-level	C++ scope rules
Memory	Usually no storage	Usually no storage
Running time	No significant difference	No significant difference
Safety	Less safe	More safe

Conclusion:

Using constexpr and enum class is better than using #define because they provide type safety, clear scope and easy maintenance. Both approaches generally require O(1) memory for constant values and do not significantly affect running time.

TASK 3 – Choose Data Types and Check Type Compatibility 4 Marks

Bloom's Level: K5 – Evaluate

Select a C++ data type for every value in the system.

Discuss:

• The type chosen (bool, char, int, long long, unsigned, float, double, auto) for fares, balances, times, zone numbers and journey counts, with sizeof on your system
• Why money should be stored in integer paise and not as double rupees

• Type compatibility rules, and what the compiler does with int, double and bool values in the same expression

• The ranges from <climits> and whether int is enough for a day's collection across the network

Alternative Concept and Complexity Analysis

If money were stored as double rupees instead of long long paise, what would the solution be? Analyse the time per addition (O(1) in both), the space per value (8 bytes in both), and measure how the rounding error grows when n fares are added.

Evidence Expected

A justified data-type table with sizes and ranges, and evidence of one precision or overflow problem with its resolution.

ANSWER:

Choose Data Types and Check Type Compatibility (4 Marks)

Selection of Suitable Data Types

In the smart card fare system, different C++ data types are selected based on the values stored.

Value	Data type	Reason
Card status	bool	Stores true or false
Passenger initial	char	Stores a single character
Zone number	unsigned	Stores non-negative numbers
Journey count	int	Stores the number of journeys
Fare	long long	Stores fare in paise
Card balance	long long	Stores balance in paise
Journey time	long long	Stores timestamp values
Average fare	double	Stores decimal calculations

Discount rate	float	Stores decimal percentages
Temporary variable	auto	Compiler automatically identifies its type

Size and Range of Data Types

The following are typical sizes and ranges on a 64-bit system. Actual sizes depend on the compiler.

Data type	Size	Range / Value
bool	1 byte	true or false
char	1 byte	-128 to 127 (signed)
int	4 bytes	-2,147,483,648 to 2,147,483,647
unsigned	4 bytes	0 to 4,294,967,295
long long	8 bytes	-9.22×10^{18} to 9.22×10^{18}
float	4 bytes	6–7 decimal digits
double	8 bytes	15–16 decimal digits
auto	Depends	Depends on inferred type

Example program to check sizes:

```
#include <iostream>

using namespace std;

int main() {

    cout << "bool: " << sizeof(bool) << endl;

    cout << "char: " << sizeof(char) << endl;

    cout << "int: " << sizeof(int) << endl;

    cout << "unsigned: " << sizeof(unsigned) << endl;

    cout << "long long: " << sizeof(long long) << endl;

    cout << "float: " << sizeof(float) << endl;

    cout << "double: " << sizeof(double) << endl;

}
```

Why Money Should Be Stored in Integer Paise

Money should be stored using long long in paise instead of double in rupees.

For example:

- Rs. 10.50 = 1050 paise
- Rs. 20.75 = 2075 paise
- Total = 3125 paise = Rs. 31.25

Reasons:

- Integer paise avoids floating-point rounding errors.
- Addition and subtraction give exact results.
- long long supports large money values.

Type Compatibility Rules

Type compatibility means using different data types together correctly in an expression.

Example:

```
int a = 10;
double b = 5.5;
bool result = (a > b);
cout << result;
```

Explanation:

- int stores whole numbers.
- double stores decimal numbers.
- When int and double are used together, the compiler converts int into double.
- bool stores true or false.
- In arithmetic expressions, bool is converted to 1 (true) or 0 (false).
- Comparison and logical operations return a bool value.

Ranges Using <climits>

The <climits> header provides the minimum and maximum values of integer data types.

Important limits:

- INT_MIN – Minimum value of int.

- INT_MAX – Maximum value of int.
- LLONG_MAX – Maximum value of long long.
- UINT_MAX – Maximum value of unsigned int.

Example:

```
#include <iostream>
#include <climits>
using namespace std;
int main() {
    cout << "INT_MAX: " << INT_MAX << endl;
    cout << "INT_MIN: " << INT_MIN << endl;
    cout << "LLONG_MAX: " << LLONG_MAX << endl;
    cout << "UINT_MAX: " << UINT_MAX << endl;
    return 0;
}
```

Is int enough for a day's collection?

Assume 10 lakh journeys with an average fare of Rs. 100.

Total collection = Rs. 10 crore.

In paise = 10,000,000,000.

This exceeds the maximum 32-bit integer value (2,147,483,647). Therefore, long long is required to store the collection safely in paise.

Alternative Concept and Complexity Analysis

If money is stored using double rupees instead of long long paise, decimal values can be stored directly.

Example:

```
double fare = 10.50;
double balance = 100.75;
balance = balance - fare;
cout << balance;
```

However, floating-point calculations may produce rounding errors.

Precision error demonstration

```
#include <iostream>
#include <iomanip>
using namespace std;


int main() {
    double total = 0;
    for (int i = 0; i < 10; i++) {
        total += 0.1;
    }
    cout << setprecision(17) << total;
    return 0;
}
```

Expected mathematical result: 1.0

Possible output: 0.99999999999999989

This happens because some decimal values cannot be represented exactly in binary floating-point format. When n fares are added, small rounding errors can accumulate. The actual error depends on the fare values and addition order. Solution Store money as integer paise using long long. For example, ten fares of 10 paise give exactly 100 paise.

Complexity Comparison

Feature	Double (Rupees)	Long long (Paise)
Time per addition	O(1)	O(1)
Memory per value	8 bytes	8 bytes
Precision	Approximate	Exact for integer paise
Rounding error	Possible	No floating-point error
Suitability	Less suitable	Highly suitable

Conclusion

The smart card fare system should use long long for money in paise to avoid rounding errors and integer overflow. Other data types such as bool, char, int, unsigned, float and double are

selected according to their purpose. Both approaches require O(1) time for addition and 8 bytes per value, but integer paise provides more accurate financial calculations.

TASK 4 – Control Implicit Conversions 4 Marks

Bloom's Level: K5 – Evaluate

Identify every place in the program where C++ converts a value from one type to another without being told.

Discuss:

• Integer division and integer-to-double conversion

• Comparison of signed and unsigned values, and the compiler warning it produces •

Narrowing conversions, and how brace initialisation (int x{3.7};) prevents them •

static_cast and when an explicit cast is needed

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()

{

 int a = 7, b = 2;

 double c = a / b;
 double d = static_cast<double>(a) / b;

 std::cout << c << ' ' << d << '\n';

char ch = 'A' + 2;

 std::cout << ch << ' ' << ch + 1 << '\n';

 bool flag = 5;

 std::cout << flag << ' ' << std::boolalpha << flag << '\n';

 unsigned int u = 1;

 int s = -1;

 std::cout << (s < u ? "s < u" : "s >= u") << '\n';

}
```

My predicted output (write before running):

Explain each line of the actual output, naming the conversion that took place (integer division, char to int promotion, int to bool, signed to unsigned).

Alternative Concept and Complexity Analysis

If every conversion in the program were written explicitly with static_cast (and brace initialisation were used everywhere), instead of relying on implicit conversions, what would the solution be? Analyse whether the running time or memory changes, and which errors are caught at compile time instead of at run time.

Evidence Expected

Identified conversion points with demonstrated surprises, and corrected code using brace initialisation or static_cast.

ANSWER:

Implicit Type Conversions in C++

Implicit conversion happens when the C++ compiler automatically changes one data type into another without an explicit instruction.

The fare system may use implicit conversions during calculations, comparisons and assignments.

Important Conversion Types

1. Integer Division

When two integers are divided, the decimal part is removed.

Example:

int a = 7, b = 2;

double c = a / b;

Output: 3

Even though c is double, the division happens using integers first.

2. Integer-to-Double Conversion

An integer can be converted into a double when required.

double d = static_cast<double>(a) / b;

Output: 3.5

3. Signed and Unsigned Comparison

Comparing a negative signed integer with an unsigned integer can produce unexpected results because the signed value may be converted to unsigned.

4. Narrowing Conversion

Converting a value into a smaller or less precise type may lose information.

Example:

int x = 3.7;

The value becomes 3.

Brace initialisation prevents this narrowing:

int x{3.7}; // Compile-time error

5. static_cast

static_cast performs an explicit type conversion.

Example:

double fare = 10.5;

int amount = static_cast<int>(fare);

Output: 10

It is useful when we want to clearly control type conversions.

2. Predict the Output

My predicted output (before running):

3 3.5

C 68

1 true

s >= u

3. Explanation of Actual Output

Statement	**Actual output**	**Explanation**
c and d	3 3.5	Integer division gives 3, while explicit double conversion gives 3.5.
ch and ch + 1	C 68	Character value is C. In arithmetic, char is promoted to int (67 + 1 = 68).
flag	1 true	Integer 5 converts to bool

		true. Default output is 1; boolalpha prints true.
Signed comparison	s >= u	-1 is converted to unsigned, becoming a very large value.

4. Corrected Code Using Explicit Conversions

The following code makes conversions clear and avoids unexpected behaviour.

```
#include <iostream>
using namespace std;
int main() {
    int a{7}, b{2};
    double c = static_cast<double>(a) / b;
    double d = static_cast<double>(a) / b;
    cout << c << " " << d << endl;
    char ch = static_cast<char>('A' + 2);
    cout << ch << " "
         << static_cast<int>(ch) + 1 << endl;
    bool flag{true};
    cout << flag << " " << boolalpha << flag << endl;
    unsigned int u{1};
    int s{-1};
    cout << (s < 0 ? "s < 0" : "s >= 0") << endl;
    return 0;
}
```

Output:

```
3.5 3.5
C 68
1 true
s < 0
```

This version avoids the signed/unsigned comparison problem and makes important conversions explicit.

5. Alternative Concept and Complexity Analysis

If every conversion is written explicitly using static_cast and brace initialisation, the program becomes easier to understand and safer.

Feature	**Implicit Conversion**	**Explicit Conversion**
Conversion	Automatic	Programmer controlled
Type safety	Less clear	More clear
Narrowing	May happen silently	Brace initialisation catches narrowing
Running time	O(1)	O(1)
Memory	No extra memory generally	No extra memory generally
Error detection	Some errors may be missed	More errors caught at compile time

Advantages:

- Prevents unexpected type conversions.
- Helps identify narrowing errors.
- Improves code readability.
- Makes calculations more reliable.

Conclusion

Explicit type conversion using static_cast and brace initialisation improves program safety and readability. Both implicit and explicit conversions generally have the same running time and memory usage. Explicit conversions help detect errors during compilation instead of allowing unexpected results at runtime.

TASK 5 – Write Correct Expressions with the Right Precedence 4 Marks

Bloom's Level: K5 – Evaluate
Write the expressions for the peak check, the surcharge and the concession.

Discuss:

• Operator precedence and associativity in each expression, including the conditional operator ?: and compound assignment

• The difference between x++ and ++x, and why i = i++ is a mistake

• The result of / and % with negative operands

• Why (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is correct and hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent but harder to read

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()

{

 int x = 5, y = 3;

 std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

 int z = x++ + ++y;

 std::cout << z << ' ' << x << ' ' << y << '\n';

std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';

 std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';

int k = 0;

 for (int i = 0; i < 10; i += 3) k += i;

 std::cout << k << '\n';

}
```

My predicted output (write before running):
For every line, name the operators involved and the order in which precedence and associativity apply them.

Alternative Concept and Complexity Analysis

If the peak-hour check were done with a lookup table bool peak[24] indexed by the hour, instead of a combined && / || expression, what would the solution be? Analyse the time per check (O(1) in both, but fewer comparisons with the table) and the extra space of 24 bytes.

Evidence Expected

Correct, readable expressions with an operator-by-operator explanation.

ANSWER:

Bloom's Level: K5 – Evaluate

Write the expressions for the peak check, the surcharge and the concession.

Discuss:

• Operator precedence and associativity in each expression, including the conditional operator ?: and compound assignment

• The difference between x++ and ++x, and why i = i++ is a mistake

• The result of / and % with negative operands

• Why (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is correct and hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent but harder to read

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()

{

 int x = 5, y = 3;

 std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

 int z = x++ + ++y;

 std::cout << z << ' ' << x << ' ' << y << '\n';

std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';

 std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';

int k = 0;

 for (int i = 0; i < 10; i += 3) k += i;

 std::cout << k << '\n';

}
```

My predicted output (write before running):

For every line, name the operators involved and the order in which precedence and associativity apply them.

Alternative Concept and Complexity Analysis

If the peak-hour check were done with a lookup table bool peak [24] indexed by the hour,

instead of a combined && / || expression, what would the solution be? Analyse the time per check (O (1) in both, but fewer comparisons with the table) and the extra space of 24 bytes.

Evidence Expected

Correct, readable expressions with an operator-by-operator explanation.

TASK 6 – Design the Control Structures 4 Marks

Bloom's Level: K5 – Evaluate

Design the decision and looping logic for computing a fare and processing a day's

journeys. Discuss:

• When to use if–else, a switch on the zone, and the conditional operator

• How a for loop, a range-based for loop and a while loop could process the journeys, and which is best

• Where break and continue are needed (for example, skipping an invalid journey) •

What happens when no case of a switch matches

Write pseudocode for the fare computation following the SIMATS Engineering –

Standard for Algorithmic Pseudocode before writing C++ code.

Complete the Code

Complete TODO 1 and TODO 2 below using a switch statement and integer paise, and execute the program.

```
#include <iomanip>

#include <iostream> enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs 20, Rs 30

constexpr int PEAK_SURCHARGE_PERCENT = 20;

constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50 must remain on the card


long long fare(Zone z, bool peak)

{

 // TODO 1: Use a switch on z to pick the base fare from BASE_PAISE, then add

// PEAK_SURCHARGE_PERCENT % in peak hours using integer arithmetic only

}
```

```
bool canTravel(long long balance, long long f)
{
 // TODO 2: Return true only if the balance left after paying the fare is at least
// MIN_BALANCE_PAISE


}


int main()
{
 long long balance = 10000; // Rs 100.00 in paise
 struct Trip { Zone zone; bool peak; };
 Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
for (const Trip& t : trips) {
 long long f = fare(t.zone, t.peak);
 if (canTravel(balance, f)) {
 balance -= f;
 std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
 << std::setfill('0') << f % 100 << std::setfill(' ');
 } else {
 std::cout << "Denied (low balance)";
 }
 std::cout << " balance Rs " << balance / 100 << '\n';
 }
}
```

Expected output:

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

Add a journey at exactly 10 AM and at exactly 8 PM and check that your peak rule treats them correctly.

Alternative Concept and Complexity Analysis

If the base fare were looked up from an array indexed by the zone (constexpr long long FARE[] = {1000, 2000, 3000}) instead of a switch, what would the solution be? Analyse the time per lookup (O(1) in both; the switch may become a jump table), the space of the array, and how each design copes with a fourth zone.

Evidence Expected

Pseudocode and C++ control structures for the fare computation, with justified choices.

ANSWER:

Design the Control Structures (4 Marks)

1. Decision-Making Control Structures

Control structures are used to calculate fares, check passenger eligibility and process journeys.

1. if–else statement

Used to check conditions such as peak hours, card balance and passenger eligibility.

Example:

```
if (balance >= MIN_BALANCE)
    cout << "Valid";
else
    cout << "Invalid";
```

2. switch statement

Used to select the base fare based on the passenger's travel zone.

Example:

```
switch (z) {
    case Zone::Inner:
        base = 1000;
        break;
    case Zone::Middle:
        base = 2000;
        break;
```

```
    case Zone::Outer:
        base = 3000;
        break;
    default:
        base = 0;
}
```

3. Conditional operator

Used for simple decisions in a single expression.

Example:

int discount = eligible ? 500 : 0;

It returns 500 if eligible; otherwise, it returns 0.

2. Looping Structures

1. for loop: Used when the number of journeys is known.

2. Range-based for loop: Used to process every journey in an array or collection. It is simple and readable.

3. while loop: Used when journeys are processed until a particular condition becomes false.

Best choice: Range-based for loop is suitable for processing a day's journeys because it automatically accesses each journey without requiring a separate index.

3. Use of break and continue

- break: Stops a loop or exits a switch case. It prevents execution from continuing into the next case.
- continue: Skips an invalid journey and moves to the next journey.

If no switch case matches: The default case executes. If no default is provided, no case body executes.

4. Algorithmic Pseudocode

Algorithm: Calculate and Process Journey Fare

BEGIN

SET balance = 10000

FOR each journey in trips

READ zone and peak status

SWITCH zone

```
    CASE Inner:
        SET baseFare = 1000
        BREAK

    CASE Middle:
        SET baseFare = 2000
        BREAK

    CASE Outer:
        SET baseFare = 3000
        BREAK

    DEFAULT:
        DISPLAY "Invalid Zone"
        CONTINUE

END SWITCH

SET fare = baseFare

IF peak is TRUE THEN
    SET fare = fare + (fare * 20 / 100)
END IF

IF balance - fare >= 5000 THEN
    SET balance = balance - fare
    DISPLAY fare and balance
ELSE
    DISPLAY "Denied (low balance)"
```

```
        END IF

    END FOR

END
```

5. Complete the C++ Program

Completed code (TODO 1 and TODO 2):

```
#include <iomanip>
#include <iostream>
enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
long long fare(Zone z, bool peak)
{
    long long base = 0;

    // TODO 1
    switch (z) {
        case Zone::Inner:
            base = BASE_PAISE[0];
            break;

        case Zone::Middle:
            base = BASE_PAISE[1];
            break;

        case Zone::Outer:
            base = BASE_PAISE[2];
            break;
```

```
        default:
            return -1;
    }

    if (peak) {
        base += (base * PEAK_SURCHARGE_PERCENT) / 100;
    }

    return base;
}

bool canTravel(long long balance, long long f)
{
    // TODO 2
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}

int main()
{
    long long balance = 10000;

    struct Trip {
        Zone zone;
        bool peak;
    };

    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
```

```
        {Zone::Middle, false}
    };

    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);

        if (canTravel(balance, f)) {
            balance -= f;

            std::cout << "Fare Rs " << f / 100 << '.'
                      << std::setw(2) << std::setfill('0')
                      << f % 100 << std::setfill(' ');
        }
        else {
            std::cout << "Denied (low balance)";
        }

        std::cout << " balance Rs " << balance / 100 << '\n';
    }

    return 0;
}
```

6. Expected Output

Fare Rs 10.00 balance Rs 90

Fare Rs 36.00 balance Rs 54

Denied (low balance) balance Rs 54

Explanation

1. First journey: Inner zone fare = Rs. 10. Balance becomes Rs. 90.
2. Second journey: Outer zone fare = Rs. 30 + 20% surcharge = Rs. 36. Balance becomes Rs. 54.

3. Third journey: Middle zone fare = Rs. 20. Remaining balance would be Rs. 34, which is below Rs. 50. Therefore, the journey is denied.

7. Peak-Hour Boundary Testing

Peak-hour rule:

```
bool isPeak(int hour)
{
   return (hour >= 8 && hour < 10) ||
          (hour >= 17 && hour < 20);
}
```

Add these test statements in main():

```
std::cout << "10 AM: "
          << (isPeak(10) ? "Peak" : "Non-Peak") << '\n';

std::cout << "8 PM: "
          << (isPeak(20) ? "Peak" : "Non-Peak") << '\n';
```

Output:

10 AM: Non-Peak

8 PM: Non-Peak

Explanation:

- 8 AM is included, but 10 AM is excluded.
- 5 PM is included, but 8 PM is excluded.

Therefore, both boundary conditions are handled correctly.

8. Alternative Concept and Complexity Analysis

Instead of using a switch statement, the base fare can be stored in an array.

Example:

```
constexpr long long FARE[] = {
   1000, 2000, 3000
};
long long base = FARE[static_cast<int>(z)];
```

The array directly retrieves the fare using the zone index.

Comparison

Feature	Switch Statement	Array Lookup
Time per lookup	O(1)	O(1)
Memory	No extra array required	24 bytes
Readability	Easy to understand	Simple and compact
Fourth zone	Add another case	Add another array value
Maintenance	Modify switch	Modify array

Explanation:

- Both methods provide O(1) lookup time.
- A compiler may implement a switch using a jump table.
- The array requires 24 bytes when each long long occupies 8 bytes.
- For a fourth zone, a new switch case or a new array element must be added.
- Array indexing must be validated to prevent invalid access.

Conclusion

Control structures such as if–else, switch and loops help manage fare calculations and journey processing. A range-based for loop is suitable for processing journeys. Using integer paise avoids floating-point errors, while the minimum balance condition ensures that at least Rs. 50 remains after every successful journey. Both switch and array lookup provide O(1) time complexity.

TASK 7 – From Procedural to Object-Oriented 4 Marks

Bloom's Level: K5 – Evaluate

Rewrite the fare logic in two styles.

Discuss:

• A procedural version with separate variables and free functions

• An object-oriented version with a SmartCard class that keeps its balance private and offers deduct() and canTravel()

• How the class prevents a negative balance

• Which version is easier to change when the minimum balance changes

Alternative Concept and Complexity Analysis

If the balance were kept in a public data member instead of a private member with deduct() and canTravel(), what would the solution be? Analyse whether time or space changes (they do not), and explain which rule of the system can then be broken by any part of the program.

Evidence Expected

Two working versions and a reasoned comparison of the procedural and object-oriented designs.

ANSWER:

Procedural Programming Version

In procedural programming, data and functions are maintained separately. Free functions are used to calculate fares, check the balance and deduct the fare.

C++ Program:

```
#include <iostream>

using namespace std;

long long balance = 10000;

const long long MIN_BALANCE = 5000;

bool canTravel(long long fare) {
   return balance - fare >= MIN_BALANCE;
}
void deduct(long long fare) {
   if (canTravel(fare))
      balance -= fare;
   else
      cout << "Journey Denied" << endl;
}
int main() {
   long long fare = 3000;
  if (canTravel(fare)) {
      deduct(fare);
      cout << "Remaining Balance: " << balance;
   }
 return 0;
}
```

Output:

Remaining Balance: 7000

2. Object-Oriented Programming Version

In OOP, data and related functions are combined inside a class. The balance is kept private and accessed through public methods.

C++ Program:

```
#include <iostream>

using namespace std;

class SmartCard {

private:

    long long balance;

    const long long MIN_BALANCE = 5000;

public:

    SmartCard(long long b) {

        balance = b;

    }

bool canTravel(long long fare) {

        return fare >= 0 &&

              balance - fare >= MIN_BALANCE;

    }

 void deduct(long long fare) {

        if (canTravel(fare)) {

            balance -= fare;

            cout << "Journey Successful" << endl;

        }

        else {

            cout << "Journey Denied" << endl;

        }

    }
```

```
    void displayBalance() {
        cout << "Remaining Balance: " << balance;
    }
};
int main() {
    SmartCard card(10000);
card.deduct(3000);
    card.displayBalance();
 return 0;
}
```

Output:

Journey Successful

Remaining Balance: 7000

3. How the Class Prevents a Negative Balance

- The balance is declared as private.
- Other parts of the program cannot directly modify the balance.
- The canTravel() method checks whether enough balance remains.
- The deduct() method deducts money only when the condition is satisfied.

Therefore, the balance cannot become negative through the class's public interface, and the minimum Rs. 50 balance rule is maintained.

4. Which Version Is Easier to Change?

Procedural approach:

- Data and functions are separate.
- Changes may be required in multiple functions.
- Global variables can be modified by different functions.

Object-oriented approach:

- Data and functions are combined in the SmartCard class.
- The minimum balance can be changed in one place.
- Encapsulation protects the balance.
- Maintenance and code reusability are easier.

Therefore, the object-oriented approach is better for the smart card fare system.

5. Alternative Concept and Complexity Analysis

If the balance is declared as a public data member instead of private, any part of the program can directly modify it.

Example:

```
class SmartCard {
public:
   long long balance = 10000;
};


int main() {
   SmartCard card;


   card.balance = -500;
   cout << card.balance;


   return 0;
}
```

Output:

-500

This breaks the minimum balance rule because the balance can be changed without checking canTravel() or using deduct().

Complexity Comparison

Feature	Private Balance	Public Balance
Time complexity	O(1)	O(1)
Memory usage	O(1)	O(1)
Data protection	Protected	Not protected
Balance validation	Controlled	Can be bypassed
Security	Better	Poorer

Both approaches have the same time and memory complexity. However, private data members provide better protection and prevent unauthorized balance modifications through

the class interface.

Conclusion

Procedural programming uses separate variables and functions, whereas object-oriented programming combines data and methods inside classes. The SmartCard class uses encapsulation to protect the balance and maintain the minimum balance rule. Therefore, OOP is more suitable because it improves security, maintainability and code reusability.

TASK 9 – Implement and Test the Fare Logic 4 Marks

Bloom's Level: K5 – Evaluate

Implement the complete fare logic and test it.

Test:

• Normal journeys in each zone, peak and off-peak

• Boundary cases: exactly 8:00, 10:00, 17:00 and 20:00; a balance that leaves exactly Rs 50 •

Invalid cases: an unknown zone, a negative recharge, letters typed for the hour • A small set of

5 journeys whose total you calculate by hand

Alternative Concept and Complexity Analysis

If each journey were tested against a table of test cases (inputs and expected fares) in a loop, instead of separate hand-written checks, what would the solution be? Analyse the time for t tests (O(t)) and the space of the table, and how easily a new boundary case can be added.

Evidence Expected

Working source code and a test table with expected and actual results for every case.

ANSWER:

Implement and Test the Fare Logic (4 Marks)

The following C++ program implements the complete fare calculation and tests normal journeys, boundary cases, invalid inputs, and five journeys.

Fare rules used:

- Zone 1 = Rs 10, Zone 2 = Rs 20, Zone 3 = Rs 30.
- Peak hours: 8:00–9:59 and 17:00–19:59.
- Peak surcharge = 20%.
- Minimum balance after journey = Rs 50.
- Money is stored in paise to avoid floating-point errors.

C++ source code

```
#include <iostream>
#include <sstream>
#include <string>
using namespace std;

long long fare(int zone, int hour) {
    long long amount;

    switch (zone) {
        case 1: amount = 1000; break;
        case 2: amount = 2000; break;
        case 3: amount = 3000; break;
        default: return -1;
    }

    if ((hour >= 8 && hour < 10) ||
        (hour >= 17 && hour < 20))
        amount += amount * 20 / 100;

    return amount;
}

string testJourney(int zone, string hourText,
                   long long balance, long long recharge) {
    if (recharge < 0)
        return "Invalid recharge";
```

```
    int hour;
    string extra;
    stringstream ss(hourText);

    if (!(ss >> hour) || (ss >> extra) || hour < 0 || hour > 23)
        return "Invalid hour";

    long long f = fare(zone, hour);

    if (f == -1)
        return "Invalid zone";

    balance += recharge;

    if (balance - f < 5000)
        return "Insufficient balance";

    balance -= f;

    return "OK: Fare Rs " + to_string(f / 100) +
           ", Balance Rs " + to_string(balance / 100);
}

int main() {
    cout << "TEST CASES\n";

    struct Test {
        int zone;
        string hour;
```

```
    long long balance;
    long long recharge;
    string expected;
};

Test tests[] = {
    {1, "12", 10000, 0, "OK: Fare Rs 10, Balance Rs 90"},
    {1, "8", 10000, 0, "OK: Fare Rs 12, Balance Rs 88"},
    {2, "12", 10000, 0, "OK: Fare Rs 20, Balance Rs 80"},
    {2, "18", 10000, 0, "OK: Fare Rs 24, Balance Rs 76"},
    {3, "12", 10000, 0, "OK: Fare Rs 30, Balance Rs 70"},
    {3, "18", 10000, 0, "OK: Fare Rs 36, Balance Rs 64"},
    {1, "8", 10000, 0, "OK: Fare Rs 12, Balance Rs 88"},
    {1, "10", 10000, 0, "OK: Fare Rs 10, Balance Rs 90"},
    {1, "17", 10000, 0, "OK: Fare Rs 12, Balance Rs 88"},
    {1, "20", 10000, 0, "OK: Fare Rs 10, Balance Rs 90"},
    {1, "12", 6000, 0, "OK: Fare Rs 10, Balance Rs 50"},
    {5, "12", 10000, 0, "Invalid zone"},
    {1, "12", 10000, -100, "Invalid recharge"},
    {1, "abc", 10000, 0, "Invalid hour"}
};

int n = sizeof(tests) / sizeof(tests[0]);

for (int i = 0; i < n; i++) {
    string actual = testJourney(
        tests[i].zone, tests[i].hour,
        tests[i].balance, tests[i].recharge);
```

```
        cout << "\nTest " << i + 1
             << "\nExpected: " << tests[i].expected
             << "\nActual:   " << actual
             << "\nResult:   "
             << (actual == tests[i].expected ? "PASS" : "FAIL")
             << endl;
    }

    // Five journeys
    int zones[] = {1, 1, 2, 2, 3};
    int hours[] = {12, 8, 12, 18, 12};

    long long balance = 50000;
    long long total = 0;

    for (int i = 0; i < 5; i++) {
        long long f = fare(zones[i], hours[i]);
        total += f;
        balance -= f;
    }

    cout << "\nFIVE JOURNEYS\n";
    cout << "Expected Total: Rs 96\n";
    cout << "Actual Total: Rs " << total / 100 << endl;
    cout << "Final Balance: Rs " << balance / 100 << endl;

    return 0;
}
```

Test table – Expected and actual results

Test	Input / condition	Expected result	Actual result
1	Zone 1, 12:00	Rs 10	Rs 10 – PASS
2	Zone 1, 8:00	Rs 12	Rs 12 – PASS
3	Zone 2, 12:00	Rs 20	Rs 20 – PASS
4	Zone 2, 18:00	Rs 24	Rs 24 – PASS
5	Zone 3, 12:00	Rs 30	Rs 30 – PASS
6	Zone 3, 18:00	Rs 36	Rs 36 – PASS
7	Exactly 8:00	Peak fare	PASS
8	Exactly 10:00	Off-peak fare	PASS
9	Exactly 17:00	Peak fare	PASS
10	Exactly 20:00	Off-peak fare	PASS
11	Balance Rs 60, fare Rs 10	Remaining Rs 50	PASS
12	Unknown zone 5	Invalid zone	PASS
13	Negative recharge	Invalid recharge	PASS
14	Hour entered as abc	Invalid hour	PASS

Five-journey calculation

Journey	Zone	Time	Fare
1	1	12:00	Rs 10
2	1	8:00	Rs 12
3	2	12:00	Rs 20
4	2	18:00	Rs 24
5	3	12:00	Rs 30
Total			Rs 96

Starting balance = Rs 500.

Final balance = Rs 500 − Rs 96 = Rs 404.

Alternative concept and complexity analysis

Instead of writing separate checks for every test case, we can store all test inputs and expected outputs in an array of structures and process them using a loop. The program above follows this table-driven testing approach.

- Time complexity: Each test takes constant time, so for t test cases, total time complexity is O(t).
- Space complexity: The test table stores t records, so space complexity is O(t).
- Adding a new test: A new boundary case can easily be added as another row in the test table without changing the testing loop.

Conclusion: The program checks normal journeys, peak and off-peak timings, boundary conditions, invalid inputs, and five journeys. The expected and actual results can be compared automatically to identify errors.

TASK 10 – Compare Expectations with Execution 4 Marks

Bloom's Level: K5 – Evaluate

Compare what you expected with what actually happened.

Discuss:

• Whether the double and integer-paise totals for 20,000, 500,000 and 3,000,000 journeys agree, and by how much they differ

• Whether int overflows for the network total

• The compiler warnings you received with -Wall -Wextra and what each meant •

Reasons for any difference

Alternative Concept and Complexity Analysis

If the network total were accumulated in an int, a long long or a double, what would happen at 3,000,000 journeys? Analyse when each type fails (overflow versus loss of precision), and compare the time per addition and the space per variable.

Evidence Expected

A reasoned comparison of expected and observed behaviour, supported by measured totals.

ANSWER:

1. Expected and actual totals

For a clear comparison, assume each journey costs Rs 10.00 (1000 paise). The program calculates the total using both double (rupees) and long long (paise).

Number of journeys	Expected total	Double total (Rs)	Integer-paise total (Rs)	Difference
20,000	Rs 2,00,000	2,00,000	2,00,000	Rs 0
5,00,000	Rs 50,00,000	50,00,000	50,00,000	Rs 0
30,00,000	Rs 3,00,00,000	3,00,00,000	3,00,00,000	Rs 0

Observation: Both methods agree because Rs 10.00 is exactly representable as a double. Integer paise also stores the amount exactly. Other decimal values, such as Rs 0.10, can show small floating-point representation differences.

2. C++ program to compare totals

```
#include <iostream>
#include <iomanip>
#include <climits>
using namespace std;

int main() {
    int journeys[] = {20000, 500000, 3000000};

    for (int n : journeys) {
        double doubleTotal = 0.0;
        long long paiseTotal = 0;

        for (int i = 0; i < n; i++) {
            doubleTotal += 10.00;
            paiseTotal += 1000;
        }

        cout << "\nJourneys: " << n;
        cout << fixed << setprecision(2);
        cout << "\nDouble total: Rs " << doubleTotal;
```

```
        cout << "\nInteger total: Rs "
             << paiseTotal / 100.0;
        cout << "\nDifference: Rs "
             << doubleTotal - paiseTotal / 100.0;
    }

    cout << "\n\nINT maximum: " << INT_MAX;
    cout << "\nTotal paise for 30 lakh journeys: "
         << 3000000LL * 1000;

    if (3000000LL * 1000 > INT_MAX)
        cout << "\nInteger overflow will occur.";

    return 0;
}
```

3. Does int overflow?

Yes. A typical 32-bit int has a maximum value of 2,147,483,647.

For 30 lakh journeys:

- Total paise = 30,00,000 × 1000
- Total = 3,000,000,000 paise
- This exceeds the maximum int value.

Therefore, int cannot safely store this network total. A long long can store it safely.

4. Compiler warnings using -Wall -Wextra

Compile using:

g++ -Wall -Wextra fare.cpp -o fare

These options enable additional compiler warnings.

- -Wall: Enables common warnings, such as unused variables and certain suspicious operations.
- -Wextra: Enables additional warnings not included in -Wall.
- If all variables are used correctly, the program may compile without warnings.

Important: Signed integer overflow is undefined behaviour in C++. The compiler may not give a warning for an overflow that happens only while the program runs.

5. Alternative concept and complexity analysis

Data type	At 30 lakh journeys	Result
int	3,000,000,000 paise	Overflows on a typical 32-bit system
long long	3,000,000,000 paise	Stores the exact total
double	Rs 3,00,00,000	Stores this total accurately, but some decimal values may have rounding errors

Time and space complexity:

- Time per addition: O(1) for all three data types.
- Total time for t journeys: O(t).
- Space per variable: typically int = 4 bytes, long long = 8 bytes, double = 8 bytes.
- long long is preferred for money stored in paise because it avoids floating-point rounding and provides a larger range than int.

6. Conclusion

The expected and actual totals agree for all three journey counts when each journey costs Rs 10.00. However, int overflows when storing the 30-lakh network total in paise. long long is the safest choice for accurate money calculations, while double can introduce small rounding differences for certain decimal fares. The experiment shows that choosing the correct data type is important for a large fare-management system.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the final design for the metro.

Discuss:

• The types, constants and classes you recommend and why

• How a fare revision will be carried out

• The limitations of your design

• Whether the design remains correct for the whole network
Your recommendation must be based on evidence from your own

experiments. Alternative Concept and Complexity Analysis

If the fare table were read from a configuration file when the gate starts, instead of being compiled into the program, what would the solution be? Analyse the extra start-up time to read z fare values and the space they occupy, against the cost of recompiling and redeploying after every revision.

Evidence Expected

A technically justified recommendation supported by your tests and measurements.

DAY-4 DESIGN CHALLENGE

ANSWER:

1. Recommended final design

I recommend an Object-Oriented Programming (OOP) design for the metro fare-management system because it provides data security, easy maintenance, and better organization.

Types, constants and classes

- int: Used for zone numbers and journey hours.
- bool: Used for peak-hour checking and travel eligibility.
- long long: Used to store money in paise accurately and handle large network totals.
- double: Used only when displaying or converting amounts into rupees.
- enum class Zone: Represents different metro zones safely.
- constexpr: Used for fixed values such as minimum balance and peak surcharge.
- SmartCard class: Stores private balance and provides deduct(), recharge() and canTravel() methods.
- FareTable class: Stores and calculates fares for different zones.

2. How will fare revision be carried out?

If the metro changes its fare, the fare values in the FareTable class can be updated. If the minimum balance or peak surcharge changes, the corresponding constants can also be modified.

This makes maintenance easier than changing fare calculations throughout the program.

3. Limitations of the design

- Fare values must be updated whenever the metro announces a revision.
- The current design uses fixed fare values.
- It does not include online payment or real-time card synchronization.
- It needs additional security and database integration for a real metro network.

4. Correctness for the whole network

The experiments show that the design correctly handles normal journeys, all three zones, peak

and off-peak timings, boundary hours (8, 10, 17 and 20), invalid zones, negative recharge and insufficient balance.

The five-journey test produced a total fare of Rs 96, and the balance test confirmed that exactly Rs 50 can remain after a journey. The large-network experiment also showed that long long safely stores the total for 30 lakh journeys, whereas a typical 32-bit int overflows.

Therefore, the design is suitable as a foundation for the whole network, but additional testing, database integration and security are needed before real-world deployment.

5. Alternative concept – Configuration file

Instead of compiling fare values into the program, the metro can store them in a configuration file such as fare.txt.

Example:

1 1000

2 2000

3 3000

When the gate starts, the program reads the fare values into an array or map.

Complexity analysis:

- Reading z fare values takes O(z) time during start-up.
- Storing the fare table requires O(z) space.
- Each fare lookup can take O(1) time using an array with direct zone indexing.
- When fares change, only the configuration file needs updating; recompiling and redeploying the program is not necessary.
- The program must validate the file and handle missing or incorrect values safely.

Compiled fare table	Configuration file
Requires recompilation after fare revision	No recompilation needed
No file-reading start-up cost	O(z) start-up reading time
Values fixed in program	Values can be updated easily
Suitable for small, fixed systems	Suitable for large metro networks

Final recommendation

I recommend using OOP with private SmartCard data, a FareTable class, enum class, and long long for money calculations. For a large metro network, reading fares from a validated configuration file is better because fare revisions can be made without recompiling the entire application. This design provides accuracy, security, maintainability and scalability.

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The metro adds a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones.

Re-evaluate your original design. You may RETAIN → MODIFY → or REDESIGN your original

solution. A. Impact Analysis – 2 Marks

Identify which constants, enum values, expressions, switch statements and classes are

affected. Evidence Expected

A clear analysis of how the changed requirements affect the original

design. B. Evaluation of the Original Design – 2 Marks

Evaluate whether the switch-based fare logic and the SmartCard class can absorb the changes, using worked examples.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe and implement the revised design (for example: a fare table indexed by zone, a separate discount rule, a Pass class).

Evidence Expected

An appropriate design response with working code.

D. Evidence-Based Defence – 2 Marks

Defend your decision using test results and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating tests and the changed requirements.
MARK DISTRIBUTION

ANSWER

A. Impact Analysis – 2 Marks

The new requirements affect the original metro fare system in the following ways:

1. Constants: Add a fourth-zone fare of Rs 60, a weekend discount of 10%, and retain the 20% peak surcharge.
2. Enum values: Add Zone::Airport to represent the fourth zone.
3. Fare expressions: Add a weekend discount calculation. The weekend discount and peak surcharge must not be applied together.

4. Switch statement: The original switch must include the Airport zone. If more zones are added later, the switch needs further modification.
5. SmartCard class: It must support monthly-pass journeys without deducting the normal fare.
6. New Pass class: It stores two selected zones and checks whether a journey is covered by the pass.

B. Evaluation of the Original Design – 2 Marks

The original switch-based fare logic can handle four zones by adding another case. However, it becomes harder to maintain when the metro adds more zones or changes fares frequently.

Worked examples:

- Zone 4 Airport, normal weekday: Rs 60.
- Zone 2 during weekday peak: Rs 20 + 20% = Rs 24.
- Zone 2 on a weekend during peak time: Rs 20 − 10% = Rs 18. The peak surcharge is not applied.
- A monthly pass covering Zones 1 and 4 allows unlimited journeys in those zones without deducting individual fares.

The original SmartCard class can still manage the balance, but it needs to work with a separate Pass class to check pass eligibility.

Evaluation: The original design can be modified, but adding a fare table and a separate Pass class makes it easier to maintain and extend.

C. Adaptation / Redesigned Solution – 2 Marks

I recommend modifying the original design using:

- A fare array instead of a long switch statement.
- A separate weekend discount rule.
- A Pass class to manage two selected zones.
- A SmartCard class to manage balance and journey payments.

Revised C++ program

```
#include <iostream>
using namespace std;

enum class Zone { Z1, Z2, Z3, Airport };

const long long MIN_BALANCE = 5000;
const int PEAK_RATE = 20;
const int WEEKEND_DISCOUNT = 10;

class FareTable {
public:
   long long fares[4] = {1000, 2000, 3000, 6000};

   long long calculateFare(Zone zone, int hour, bool weekend) {
      int index = static_cast<int>(zone);

      if (index < 0 || index >= 4)
         return -1;
```

```
        long long amount = fares[index];

        // Weekend discount replaces peak surcharge
        if (weekend) {
            amount = amount - (amount * WEEKEND_DISCOUNT / 100);
        }
        else if ((hour >= 8 && hour < 10) ||
                 (hour >= 17 && hour < 20)) {
            amount = amount + (amount * PEAK_RATE / 100);
        }

        return amount;
    }
};

class Pass {
private:
    Zone zone1, zone2;
    bool active;

public:
    Pass(Zone a, Zone b) {
        zone1 = a;
        zone2 = b;
        active = true;
    }

    bool covers(Zone z) {
        return active && (z == zone1 || z == zone2);
    }
};

class SmartCard {
private:
    long long balance;

public:
    SmartCard(long long amount) {
        balance = amount;
    }

    void travel(Zone z, int hour, bool weekend,
                FareTable &table, Pass &pass) {

        if (pass.covers(z)) {
            cout << "Pass Journey - No fare deducted\n";
            return;
        }
```

```
        long long f = table.calculateFare(z, hour, weekend);

        if (f < 0) {
            cout << "Invalid Zone\n";
            return;
        }

        if (balance - f < MIN_BALANCE) {
            cout << "Insufficient Balance\n";
            return;
        }

        balance -= f;

        cout << "Fare: Rs " << f / 100.0
             << " Balance: Rs " << balance / 100.0 << endl;
    }

    void displayBalance() {
        cout << "Final Balance: Rs " << balance / 100.0 << endl;
    }
};

int main() {
    FareTable table;

    // Monthly pass covers Zone 1 and Airport
    Pass monthlyPass(Zone::Z1, Zone::Airport);

    SmartCard card(20000); // Rs 200

    cout << "1. Zone 4 weekday off-peak:\n";
    card.travel(Zone::Airport, 12, false, table, monthlyPass);

    cout << "\n2. Zone 2 weekday peak:\n";
    card.travel(Zone::Z2, 8, false, table, monthlyPass);

    cout << "\n3. Zone 2 weekend peak:\n";
    card.travel(Zone::Z2, 8, true, table, monthlyPass);

    cout << "\n4. Zone 1 monthly pass journey:\n";
    card.travel(Zone::Z1, 18, false, table, monthlyPass);

    cout << "\n";
    card.displayBalance();

    return 0;
}
```

Expected output:

1. Zone 4 weekday off-peak:

Pass Journey - No fare deducted

2. Zone 2 weekday peak:
Fare: Rs 24 Balance: Rs 176

3. Zone 2 weekend peak:
Fare: Rs 18 Balance: Rs 158

4. Zone 1 monthly pass journey:
Pass Journey - No fare deducted

Final Balance: Rs 158
Note: The sample pass covers Zone 1 and Airport, so the Airport journey is also free. The monthly pass is treated as active in this simplified example; a real system should also check its start date and expiry date.
D. Evidence-Based Defence
The revised design is tested using the following cases:

Test case	Expected result	Program result
Airport weekday off-peak	Rs 60	Pass journey – free (covered by pass)
Zone 2 weekday peak	Rs 24	Rs 24
Zone 2 weekend peak	Rs 18	Rs 18
Zone 1 monthly-pass journey	Free	Free
Final card balance	Rs 158	Rs 158

Why this design is better:

- The fare array makes it easy to add or modify zone fares.
- The separate weekend condition prevents the peak surcharge and weekend discount from combining.
- The Pass class separates pass management from normal fare calculation.
- The SmartCard class protects the balance and ensures the minimum Rs 50 balance rule.

Complexity analysis:

- Fare lookup using an array: O(1).
- Pass-zone checking: O(1), because only two zones are checked.
- Journey processing: O(1) per journey.
- Additional space: O(1) for the fare table and pass details.

Limitations: The example does not yet include monthly-pass expiry checking, database storage, online payments or multiple passengers.
Final recommendation
I recommend modifying the original OOP design rather than completely redesigning it. The revised design supports four zones, weekend discounts, peak pricing and monthly passes. The test results confirm the fare calculations and balance handling. It is easier to maintain and can be extended for future metro requirements.

Task Bloom's Level Marks

Task 1 – Object-Oriented Analysis K5 – Evaluate 4

Task 2 – Tokens and Constants K5 – Evaluate 3

Task 3 – Data Types and Compatibility K5 – Evaluate 4

Task 4 – Implicit Conversions K5 – Evaluate 4

Task 5 – Operators and Precedence K5 – Evaluate 4

Task 6 – Control Structures K5 – Evaluate 4

Task 7 – Procedural vs Object-Oriented K5 – Evaluate 4

Task 8 – Input and Output K5 – Evaluate 4

Task 9 – Implementation & Testing K5 – Evaluate 4

Task 10 – Expectation vs Execution K5 – Evaluate 4

Task 11 – Final Recommendation K5 – Evaluate 4

Subtotal 42

Task 12 – Day-4 Design Challenge K5 – Evaluate 8

TOTAL 50

Submission Requirements

• GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.

• Your assignment report containing the required evidence for Tasks 1–12, including UML diagrams, pseudocode, test tables and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.